Omnix Radiance v0.5

Radiance Virtual Geometry Implementation Tasklist

Document Purpose

This document defines the complete implementation plan for **Radiance Virtual Geometry**, abbreviated as **RVG**.

RVG is the virtualized geometry subsystem of Omnix Radiance. Its purpose is to render extremely dense static geometry using:

- Offline cluster generation.
- Hierarchical geometric LOD.
- GPU-driven visibility.
- Cluster-level frustum, cone and occlusion culling.
- Fine-grained geometry streaming.
- Persistent GPU scene data.
- Visibility-buffer rasterization.
- Deferred material evaluation.
- Hardware and optional software rasterization.
- Conventional geometry fallback.

RVG must be developed incrementally. Every milestone must leave the engine in a buildable and visually testable state.

---

1. Global Engineering Rules

These rules apply to every task and every coding agent.

1.1 Correctness before optimization

- Do not introduce a faster path until a slower reference path works.
- The conventional CPU-driven renderer remains the visual reference.
- Every new rendering path must be compared against the reference path.
- GPU rendering must never silently produce missing geometry.
- Missing detailed pages must fall back to resident parent geometry.
- Buffer overflow must generate diagnostics instead of memory corruption.

1.2 Hybrid renderer requirement

RVG must coexist with the conventional renderer.

The conventional path remains responsible for:

- Transparent materials.
- Skeletal meshes.
- Morph targets.
- Cloth.
- Runtime-deformed meshes.
- Particles.
- Hair.
- Unsupported materials.
- Collision/debug meshes.
- RVG fallback rendering.
- Unsupported GPU hardware.

1.3 Ownership rules

- ECS components own lightweight asset and instance handles.
- Asset registries own CPU asset metadata.
- `GeometryArena` owns global GPU geometry allocations.
- `GPUScene` owns persistent GPU instance records.
- `RVGResidencyManager` owns page residency state.
- No component stores raw Vulkan buffer pointers.
- No render item stores temporary pointers that survive the frame.
- Every reusable allocation must have a generation counter.

1.4 Vulkan safety rules

Every GPU pass must define:

- Input buffers and images.
- Output buffers and images.
- Required image layouts.
- Pipeline stages.
- Access masks.
- Queue ownership.
- Capacity limits.
- Overflow behaviour.
- Debug name.

Every GPU-generated work list must contain:

```
struct GPUWorkListState
{
    uint32_t count;
    uint32_t capacity;
    uint32_t overflowed;
    uint32_t droppedCount;
    uint32_t highWaterMark;
};
```

1.5 Required agent report

At the end of every milestone, produce:

- Modified files.
- New files.
- Deleted files, if any.
- Architecture changes.
- Build result.
- Runtime result.
- Validation layer result.
- Tests added.
- Known limitations.
- Performance measurements.
- Remaining blockers.
- Recommended next task.

---

2. Development Milestones

```
G0  Geometry Contracts
G1  Stable Conventional Geometry
G2  Global Geometry Arena
G3  Persistent GPU Scene
G4  GPU-Driven Indexed Rendering
G5  HZB Occlusion
G6  RVG Asset Cooker
G7  All-Resident Cluster Renderer
G8  Hierarchical LOD Selection
G9  Virtual Geometry Page Streaming
G10 Visibility Buffer
G11 Attribute Reconstruction
G12 Material Binning and Resolve
G13 Mesh Shader Backend
G14 Software Micro-Rasterizer
G15 Shadows and Multi-View Rendering
G16 Production Hardening
```

No milestone may begin until the exit gates of its dependencies pass.

---

3. G0 — Geometry Contract Foundation

Objective

Define the permanent contracts that all conventional and virtual geometry systems will follow.

Dependencies

None.

3.1 Audit the existing renderer

- [ ] Locate every mesh asset type.
- [ ] Locate every runtime mesh type.
- [ ] Locate every Vulkan vertex buffer allocation path.
- [ ] Locate every Vulkan index buffer allocation path.
- [ ] Locate all direct draw calls.
- [ ] Locate all indirect draw calls.
- [ ] Locate all mesh binding functions.
- [ ] Locate all GBuffer mesh rendering paths.
- [ ] Locate all depth-only mesh rendering paths.
- [ ] Locate all shadow mesh rendering paths.
- [ ] Locate all Object ID rendering paths.
- [ ] Locate all editor wireframe rendering paths.
- [ ] Locate all mesh lifetime and destruction paths.
- [ ] Document where CPU and GPU mesh ownership are currently mixed.
- [ ] Document where the indirect renderer assumes a global geometry buffer.
- [ ] Document all temporary compatibility hacks.

3.2 Introduce geometry handles

Create:

```
struct GeometryHandle
{
    uint32_t index;
    uint32_t generation;
};

struct VirtualGeometryHandle
{
    uint32_t index;
    uint32_t generation;
};
```

Tasks:

- [ ] Add invalid-handle representation.
- [ ] Add equality operators.
- [ ] Add hash support.
- [ ] Add generation validation.
- [ ] Add debug string output.
- [ ] Prevent implicit conversion to raw array indices.
- [ ] Add stale-handle tests.
- [ ] Add asset reload behaviour.

- [ ] Add scene unload behaviour.

3.3 Separate asset and runtime objects

Create or standardize:

```
MeshAsset
MeshRuntime
VirtualGeometryAsset
VirtualGeometryRuntime
```

`MeshAsset` contains:

- CPU vertex data.
- CPU index data.
- Submesh descriptions.
- Material slots.
- Local-space bounds.
- Source path.
- Source asset hash.
- Import settings.

`MeshRuntime` contains:

- Geometry allocation handle.
- Vertex/index offsets.
- Vertex/index counts.
- GPU residency state.
- Runtime generation.
- Fallback state.

`VirtualGeometryAsset` contains:

- Hierarchy metadata.
- Cluster descriptors.
- Page descriptors.
- Root geometry reference.
- Fallback mesh reference.
- Material slots.
- Asset bounds.
- Maximum geometric error.

3.4 Define renderer routing

Create:

```
enum class GeometryRenderingPath
{
    ConventionalCPU,
```

```
    ConventionalGPUDriven,
    VirtualGeometry,
    Fallback
};
```

Routing rules:

- [ ] Static opaque compatible mesh can use RVG.
- [ ] Transparent mesh uses conventional renderer.
- [ ] Skeletal mesh uses conventional renderer.
- [ ] Deformed mesh uses conventional renderer.
- [ ] Failed RVG asset uses fallback mesh.
- [ ] Unsupported GPU uses conventional path.
- [ ] Editor can force any compatible path.
- [ ] Debug UI displays the selected path per object.

3.5 Define capability tiers

Create runtime feature detection:

```
Tier 0: CPU-driven indexed rendering
Tier 1: GPU-driven indexed indirect rendering
Tier 2: Mesh shader cluster rendering
Tier 3: Software micro-triangle rasterization
```

Tasks:

- [ ] Query `vkCmdDrawIndexedIndirectCount` support.
- [ ] Query buffer device address support.
- [ ] Query descriptor indexing support.
- [ ] Query mesh shader extension support.
- [ ] Query subgroup feature support.
- [ ] Query required storage buffer limits.
- [ ] Query atomic capability requirements.
- [ ] Generate a startup capability report.
- [ ] Select a safe default tier.
- [ ] Allow developer override.
- [ ] Reject unsupported forced tiers with a clear error.

3.6 Documentation

Create:

- [ ] `RADIANCE_GEOMETRY_CONTRACT.md`
- [ ] `RVG_ARCHITECTURE.md`
- [ ] `RVG_CAPABILITY_TIERS.md`
- [ ] `RVG_FALLBACK_POLICY.md`
- [ ] `RVG_RESOURCE_OWNERSHIP.md`

G0 exit gates

- [ ] Renderer builds.
- [ ] Existing scenes load.
- [ ] Geometry handles survive asset reload safely.
- [ ] Stale handles resolve to invalid/fallback resources.
- [ ] Every geometry asset has a known rendering route.
- [ ] No raw Vulkan geometry resource is owned by an ECS component.
- [ ] Capability report appears at engine startup.

---

4. G1 — Stable Conventional Geometry Renderer

Objective

Create a correct reference renderer before any virtual geometry optimization.

Dependencies

G0.

4.1 Disable unsafe indirect rendering

- [ ] Set CPU-driven rendering as the default.
- [ ] Mark existing indirect rendering experimental.
- [ ] Add runtime warning when experimental mode is enabled.
- [ ] Prevent the editor from storing experimental mode as the default.
- [ ] Add clean fallback when indirect initialization fails.

4.2 Standardize render items

Create:

```
struct RenderItem
{
    uint32_t entityID;
    GeometryHandle geometry;
    uint32_t submeshIndex;
    MaterialHandle material;

    Mat4 model;
    Mat4 previousModel;

    AABB worldBounds;
    Sphere worldSphere;

    uint32_t objectID;
```

```
    uint32_t meshID;
    uint32_t materialID;
    uint32_t flags;
};
```

Tasks:

- [ ] Validate geometry handle.
- [ ] Validate submesh range.
- [ ] Resolve missing material to fallback.
- [ ] Reject zero index-count items.
- [ ] Reject NaN/Inf transforms.
- [ ] Recalculate world bounds.
- [ ] Add deterministic sorting.
- [ ] Keep opaque and transparent queues separate.

4.3 Correct mesh binding

For every CPU draw:

- [ ] Bind correct vertex buffer.
- [ ] Bind correct index buffer.
- [ ] Apply correct buffer offsets.
- [ ] Bind correct object data.
- [ ] Bind correct material.
- [ ] Draw correct index range.
- [ ] Apply correct `firstIndex`.
- [ ] Apply correct `vertexOffset`.
- [ ] Apply correct `instanceCount`.
- [ ] Validate index range against allocation.

Apply this to:

- [ ] Depth prepass.
- [ ] GBuffer pass.
- [ ] Object ID pass.
- [ ] Mesh ID pass.
- [ ] Shadow pass.
- [ ] Wireframe mode.
- [ ] Selection outline pass.

4.4 Fallback geometry

- [ ] Create fallback cube.
- [ ] Create fallback material.
- [ ] Render invalid mesh references with fallback cube.
- [ ] Render missing material with a visible error material.
- [ ] Log asset identity once, not every frame.
- [ ] Add editor command to locate broken asset references.

4.5 Permanent geometry test scene

Create:

```
RendererTest_RVG.omnixscene
```

Include:

- Cube.
- Sphere.
- Plane.
- High-poly sphere.
- Imported OBJ.
- Imported glTF.
- Multiple instances of one mesh.
- Multiple different meshes.
- Multiple material slots.
- Negative scale test.
- Non-uniform scale test.
- Very large object.
- Very small object.
- Near-plane intersection.
- Overlapping meshes.
- Missing mesh test.
- Missing material test.

4.6 Debug modes

- [ ] Lit.
- [ ] Unlit.
- [ ] Wireframe.
- [ ] Depth.
- [ ] Linear depth.
- [ ] Object ID.
- [ ] Mesh ID.
- [ ] Material ID.
- [ ] Vertex normals.
- [ ] Tangents.
- [ ] UV checker.
- [ ] Bounds.
- [ ] Backface visualization.

4.7 Reference image tests

- [ ] Add deterministic camera transforms.
- [ ] Add screenshot capture.
- [ ] Add golden-image comparison.
- [ ] Add tolerance configuration.
- [ ] Capture CPU reference images.

- [ ] Record object and mesh ID buffers.
- [ ] Verify depth silhouette parity.

G1 exit gates

- [ ] Different meshes render correctly in one frame.
- [ ] Changing draw order does not corrupt geometry.
- [ ] Different materials render correctly.
- [ ] Depth matches visible geometry.
- [ ] Object ID appears only on mesh pixels.
- [ ] Negative and non-uniform scale do not break rendering.
- [ ] Validation layers report no geometry-related errors.
- [ ] Golden reference images are stored.

---

5. G2 — Global Geometry Arena

Objective

Make all conventional geometry globally addressable from shared GPU buffers.

Dependencies

G1.

5.1 Arena architecture

Create:

```
GeometryArena
├── GlobalVertexBuffer
├── GlobalIndexBuffer
├── VertexAllocator
├── IndexAllocator
├── StagingUploader
├── AllocationTable
├── DeferredFreeQueue
├── GrowthManager
└── Statistics
```

5.2 Allocation structures

```
struct GeometryAllocation
{
    uint64_t vertexByteOffset;
    uint64_t vertexByteSize;
```

```
    uint64_t indexByteOffset;
    uint64_t indexByteSize;

    uint32_t firstIndex;
    int32_t vertexOffset;

    uint32_t vertexCount;
    uint32_t indexCount;

    uint32_t generation;
    uint32_t flags;
};
```

Tasks:

- [ ] Implement allocation ID.
- [ ] Implement generation counter.
- [ ] Implement free-list allocator.
- [ ] Implement aligned allocation.
- [ ] Implement allocation merging.
- [ ] Implement out-of-memory response.
- [ ] Add high-water statistics.
- [ ] Add fragmentation statistics.
- [ ] Add allocation debug names.

5.3 Global vertex/index buffers

- [ ] Select initial buffer capacities.
- [ ] Use device-local memory.
- [ ] Enable transfer destination usage.
- [ ] Enable shader device address if supported.
- [ ] Enable indirect-compatible usage where necessary.
- [ ] Implement safe arena growth.
- [ ] Preserve allocations during growth.
- [ ] Replace GPU buffer references atomically.
- [ ] Defer old buffer destruction until GPU completion.

5.4 Upload path

- [ ] Create persistent staging ring.
- [ ] Add upload reservation.
- [ ] Add wraparound handling.
- [ ] Add transfer command batching.
- [ ] Add timeline semaphore tracking.
- [ ] Add upload completion state.
- [ ] Prevent rendering before upload completion.
- [ ] Add partial upload support.
- [ ] Add asset reload upload path.
- [ ] Add upload bandwidth statistics.

5.5 Deferred freeing

- [ ] Track last-used frame.
- [ ] Place freed allocations into deferred queue.
- [ ] Reclaim only after corresponding GPU fence completes.
- [ ] Prevent allocation reuse during in-flight rendering.
- [ ] Validate generation after reuse.

5.6 Convert conventional renderer

- [ ] Upload every mesh into the arena.
- [ ] Replace per-mesh vertex buffer binding.
- [ ] Replace per-mesh index buffer binding.
- [ ] Bind global buffers once per pass.
- [ ] Draw using global offsets.
- [ ] Keep optional legacy per-mesh path temporarily.
- [ ] Compare arena and legacy images.
- [ ] Remove legacy path only after parity.

5.7 Geometry arena debugging

Add panel:

- Total vertex memory.
- Total index memory.
- Used memory.
- Free memory.
- Largest free block.
- Fragmentation percentage.
- Allocation count.
- Upload bytes per frame.
- Pending uploads.
- Deferred frees.
- Growth count.

G2 exit gates

- [ ] All conventional meshes render from shared buffers.
- [ ] CPU reference images remain unchanged.
- [ ] Geometry allocations survive arena growth.
- [ ] Asset unloading does not corrupt reused allocations.
- [ ] No mesh binds an independent vertex/index buffer in the main path.
- [ ] `firstIndex` and `vertexOffset` correctly address every mesh.

---

6. G3 — Persistent GPU Scene

Objective

Represent scene instances using stable GPU records independent of temporary CPU render queues.

Dependencies

G2.

6.1 Instance allocator

Create:

```
struct GPUSceneInstanceHandle
{
    uint32_t index;
    uint32_t generation;
};
```

Tasks:

- [ ] Allocate stable instance slots.
- [ ] Recycle slots safely.
- [ ] Validate generations.
- [ ] Add free list.
- [ ] Add maximum capacity.
- [ ] Add growth behaviour.
- [ ] Add entity-to-instance lookup.
- [ ] Add instance-to-entity debug lookup.

6.2 GPU instance record

```
struct GPUGeometryInstance
{
    Mat4 model;
    Mat4 previousModel;

    Vec4 worldBoundsSphere;

    uint32_t geometryID;
    uint32_t materialTableOffset;
    uint32_t objectID;
    uint32_t flags;
};
```

Tasks:

- [ ] Confirm CPU/GPU alignment.
- [ ] Add static assertions.
- [ ] Define shader equivalent.
- [ ] Add serialization-independent layout.
- [ ] Add negative-scale flag.
- [ ] Add visible flag.
- [ ] Add cast-shadow flag.
- [ ] Add virtual-geometry flag.
- [ ] Add render-layer mask.

6.3 Dirty update system

- [ ] Track created instances.
- [ ] Track deleted instances.
- [ ] Track transformed instances.
- [ ] Track changed geometry.
- [ ] Track changed material overrides.
- [ ] Combine contiguous dirty ranges.
- [ ] Upload only changed records.
- [ ] Preserve previous transform before current update.
- [ ] Handle edit/play mode scene cloning.

6.4 GPU mesh table

```
struct GPUMeshRecord
{
    uint32_t firstIndex;
    int32_t vertexOffset;
    uint32_t indexCount;
    uint32_t vertexCount;

    Vec4 localBoundsSphere;

    uint32_t materialSlotOffset;
    uint32_t submeshOffset;
    uint32_t submeshCount;
    uint32_t flags;
};
```

Tasks:

- [ ] Upload mesh records.
- [ ] Update records after arena growth.
- [ ] Invalidate stale records.
- [ ] Add mesh generation tracking.
- [ ] Add fallback mesh record.

6.5 GPU material override table

- [ ] Store per-instance material overrides.
- [ ] Support no-override sentinel.
- [ ] Support multiple submesh material slots.
- [ ] Add update and removal path.
- [ ] Add capacity checks.

6.6 GPU scene diagnostics

Display:

- Instance count.
- Active slots.
- Free slots.
- Dirty ranges.
- Upload bytes.
- Stale-handle errors.
- GPU mesh record count.
- Material override count.

G3 exit gates

- [ ] Scene instances use stable GPU indices.
- [ ] Destroyed objects cannot be rendered accidentally.
- [ ] Moving objects updates current and previous transforms.
- [ ] Asset reload updates mesh table safely.
- [ ] CPU renderer can resolve geometry using GPU scene records.
- [ ] No frame-persistent pointer references a temporary render item.

---

7. G4 — GPU-Driven Indexed Rendering

Objective

Create a correct GPU culling and indirect draw path using global geometry buffers.

Dependencies

G2 and G3.

7.1 GPU work buffers

Create:

- [ ] Candidate instance buffer.
- [ ] Visible instance buffer.
- [ ] Indirect command buffer.

- [ ] Draw count buffer.
- [ ] Work-list state buffer.
- [ ] Debug reason buffer.
- [ ] Readback statistics buffer.

7.2 Instance frustum culling

Compute shader tasks:

- [ ] Read GPU instance.
- [ ] Skip disabled instances.
- [ ] Test render-layer mask.
- [ ] Transform or read world bounds.
- [ ] Test six frustum planes.
- [ ] Append visible instance.
- [ ] Record culled reason.
- [ ] Guard append capacity.
- [ ] Set overflow flag.

7.3 Indirect command generation

Generate:

```
VkDrawIndexedIndirectCommand
{
    indexCount,
    instanceCount,
    firstIndex,
    vertexOffset,
    firstInstance
};
```

Tasks:

- [ ] Resolve mesh record.
- [ ] Resolve correct index count.
- [ ] Resolve global offsets.
- [ ] Store GPU instance index in `firstInstance`.
- [ ] Generate one command per visible submesh.
- [ ] Support material slot resolution.
- [ ] Guard command capacity.
- [ ] Write final draw count.

7.4 Indirect execution

- [ ] Insert compute-to-draw-indirect barrier.
- [ ] Bind global vertex/index buffers.
- [ ] Bind GPU scene descriptors.
- [ ] Execute indirect-count draw.

- [ ] Add fallback when indirect-count is unavailable.
- [ ] Support depth pass.
- [ ] Support GBuffer pass.
- [ ] Support Object ID pass.
- [ ] Support shadow pass later.

7.5 CPU/GPU image parity

Create comparison modes:

- CPU path.
- GPU path.
- Split-screen.
- Difference image.
- Alternating-frame comparison.

Tests:

- [ ] One mesh.
- [ ] Multiple meshes.
- [ ] Multiple instances.
- [ ] Multiple materials.
- [ ] Negative scale.
- [ ] Frustum edge.
- [ ] Near-plane intersection.
- [ ] Large scene.
- [ ] Dynamic transforms.
- [ ] Deleted instance.

7.6 Indirect command inspector

Display:

- Command count.
- First index.
- Vertex offset.
- First instance.
- Geometry ID.
- Object ID.
- Material slot.
- Invalid command count.
- Overflow state.

G4 exit gates

- [ ] GPU and CPU paths produce equivalent images.
- [ ] Multiple mesh assets render correctly.
- [ ] Render queue order does not affect results.
- [ ] Indirect command count matches visible submesh count.
- [ ] Frustum culling never removes visible test objects.
- [ ] No validation errors.

- [ ] No indirect work-list overflow in test scenes.

---

8. G5 — Hierarchical Z-Buffer Occlusion

Objective

Introduce conservative GPU occlusion culling.

Dependencies

G4 and stable depth.

8.1 HZB image

- [ ] Create mipmapped depth image.
- [ ] Choose reversed-Z or standard-Z reduction logic.
- [ ] Document depth convention.
- [ ] Generate all mip levels.
- [ ] Add compute reduction shader.
- [ ] Add barriers between mip writes and reads.
- [ ] Handle viewport resize.
- [ ] Handle MSAA resolve if applicable.

8.2 Previous-frame depth policy

- [ ] Preserve previous valid depth.
- [ ] Detect camera cut.
- [ ] Detect camera teleport.
- [ ] Detect large FOV change.
- [ ] Detect projection-mode change.
- [ ] Detect viewport resize.
- [ ] Disable occlusion for one frame after invalidation.
- [ ] Add conservative camera-motion expansion.

8.3 Occlusion test

- [ ] Project instance sphere/AABB.
- [ ] Calculate screen rectangle.
- [ ] Select HZB mip.
- [ ] Sample conservative depth points.
- [ ] Compare nearest object depth.
- [ ] Preserve near-plane intersecting objects.
- [ ] Add configurable depth bias.
- [ ] Record occlusion result.
- [ ] Append surviving instances.

8.4 HZB debugging

Add:

- HZB mip viewer.
- Freeze HZB.
- Freeze camera.
- Occlusion bounds.
- Visible/occluded color view.
- Culling reason view.
- False-positive investigation mode.
- Previous-depth validity state.

G5 exit gates

- [ ] Occluded instances are removed.
- [ ] Visible instances do not disappear.
- [ ] Camera cuts do not create missing geometry.
- [ ] Resize and FOV changes recover safely.
- [ ] HZB output is inspectable.
- [ ] CPU and GPU paths remain visually equivalent.

---

9. G6 — RVG Offline Asset Cooker

Objective

Convert source triangle meshes into virtualized cluster hierarchies and streamable pages.

Dependencies

G0–G2. Runtime GPU virtual rendering is not required yet.

9.1 Cooker command-line application

Create:

```
OmnixRVGCooker
```

Arguments:

- Source file.
- Output `.rvg` file.
- Cluster vertex limit.
- Cluster triangle limit.
- Target hierarchy fanout.
- Simplification settings.

- Page size.
- Compression mode.
- Debug metadata option.
- Deterministic mode.
- Validation-only mode.

9.2 Mesh canonicalization

- [ ] Convert coordinate system.
- [ ] Normalize winding.
- [ ] Triangulate polygons.
- [ ] Validate indices.
- [ ] Remove invalid triangles.
- [ ] Remove duplicate triangles.
- [ ] Detect zero-area triangles.
- [ ] Generate missing normals.
- [ ] Generate missing tangents.
- [ ] Preserve UV seams.
- [ ] Preserve hard-normal seams.
- [ ] Preserve material boundaries.
- [ ] Generate deterministic vertex ordering.
- [ ] Calculate local bounds.

9.3 Adjacency construction

- [ ] Build edge map.
- [ ] Identify neighbouring triangles.
- [ ] Identify boundary edges.
- [ ] Identify material boundaries.
- [ ] Identify UV seams.
- [ ] Identify hard-normal seams.
- [ ] Detect non-manifold edges.
- [ ] Store adjacency debug data.
- [ ] Produce validation report.

9.4 Material partitioning

- [ ] Partition triangles by material.
- [ ] Prevent cluster crossing of incompatible material boundaries.
- [ ] Preserve source material slot IDs.
- [ ] Support empty material slots.
- [ ] Validate material-slot limits.
- [ ] Build per-partition bounds.

9.5 Leaf cluster generation

For each partition:

- [ ] Select initial seed triangle.
- [ ] Grow cluster using adjacency.

- [ ] Minimize boundary edge count.
- [ ] Respect vertex limit.
- [ ] Respect triangle limit.
- [ ] Create local vertex list.
- [ ] Create compact local triangle indices.
- [ ] Calculate AABB.
- [ ] Calculate bounding sphere.
- [ ] Calculate normal cone.
- [ ] Store source triangle mapping.
- [ ] Validate no triangle is lost or duplicated.

9.6 Cluster grouping

- [ ] Build cluster adjacency.
- [ ] Group neighbouring clusters.
- [ ] Keep groups spatially coherent.
- [ ] Limit group size.
- [ ] Minimize shared boundary.
- [ ] Record child-cluster list.
- [ ] Calculate group bounds.

9.7 Hierarchical simplification

For every cluster group:

- [ ] Merge child geometry.
- [ ] Mark external group boundary.
- [ ] Lock required boundary edges.
- [ ] Simplify internal geometry.
- [ ] Measure simplification error.
- [ ] Re-cluster simplified geometry.
- [ ] Generate parent representation.
- [ ] Preserve material compatibility.
- [ ] Validate boundary continuity.
- [ ] Continue recursively until root is produced.

9.8 Geometric error

- [ ] Define object-space error metric.
- [ ] Store error per hierarchy node.
- [ ] Ensure parent error is not smaller than child error.
- [ ] Add monotonicity validation.
- [ ] Add geometric-error visualizer.
- [ ] Export error histogram.
- [ ] Record triangle reduction per hierarchy level.

9.9 Root geometry

- [ ] Generate compact root representation.
- [ ] Mark root page permanently resident.

- [ ] Ensure root covers complete asset.
- [ ] Verify no holes.
- [ ] Store maximum geometric error.
- [ ] Generate root-only preview.

9.10 Conventional fallback mesh

- [ ] Generate fallback mesh.
- [ ] Select fallback detail level.
- [ ] Preserve material slots.
- [ ] Store conventional indices and vertices.
- [ ] Validate fallback independently.
- [ ] Allow editor-configured fallback complexity.

9.11 Geometry quantization

- [ ] Quantize positions relative to cluster bounds.
- [ ] Quantize normals.
- [ ] Quantize tangents.
- [ ] Quantize UVs where acceptable.
- [ ] Store quantization metadata.
- [ ] Measure maximum reconstruction error.
- [ ] Reject quantization exceeding configured tolerance.
- [ ] Add cooker comparison report.

9.12 Page packing

- [ ] Define configurable page size.
- [ ] Place cluster descriptors.
- [ ] Place vertex payloads.
- [ ] Place triangle payloads.
- [ ] Prevent cluster payload crossing invalid page boundaries.
- [ ] Record page dependencies.
- [ ] Keep parent fallback dependencies explicit.
- [ ] Pack spatially related clusters near each other.
- [ ] Generate page statistics.
- [ ] Generate page utilization histogram.

9.13 Compression

- [ ] Implement payload compression interface.
- [ ] Compress vertex data.
- [ ] Compress index data.
- [ ] Store compressed and uncompressed sizes.
- [ ] Store checksum.
- [ ] Validate decompression.
- [ ] Measure compression ratio.
- [ ] Support uncompressed debug mode.

9.14 `.rvg` file serialization

Write:

- File header.
- Section directory.
- Asset metadata.
- Material slot table.
- Hierarchy nodes.
- Cluster descriptors.
- Page descriptors.
- Page dependencies.
- Root page.
- Compressed pages.
- Fallback mesh.
- Optional debug metadata.
- Checksums.

9.15 Cooker testing

Test assets:

- Cube.
- UV sphere.
- Suzanne.
- Grid plane.
- High-poly statue.
- Scanned rock.
- Mesh with multiple materials.
- Mesh with UV seams.
- Mesh with hard normals.
- Non-manifold mesh.
- Extremely large-coordinate mesh.
- Thin geometry.
- Disconnected mesh islands.

Tests:

- [ ] Deterministic output hash.
- [ ] No triangle loss.
- [ ] No material-slot corruption.
- [ ] Hierarchy error monotonicity.
- [ ] Page checksum.
- [ ] Decode round trip.
- [ ] Root representation completeness.
- [ ] Fallback mesh validity.

G6 exit gates

- [ ] High-poly mesh cooks successfully.

- [ ] `.rvg` output is deterministic.
- [ ] Cooker can decode its own output.
- [ ] Every source region is represented by the hierarchy.
- [ ] Root geometry has no holes.
- [ ] Material boundaries are preserved.
- [ ] Error metrics are monotonic.
- [ ] Cooker produces a complete report.

---

10. G7 — All-Resident Cluster Renderer

Objective

Render RVG clusters while all geometry remains permanently resident.

Dependencies

G4 and G6.

10.1 RVG runtime loader

- [ ] Parse file header.
- [ ] Validate magic.
- [ ] Validate format version.
- [ ] Validate cooker version compatibility.
- [ ] Validate section bounds.
- [ ] Validate checksums.
- [ ] Load hierarchy metadata.
- [ ] Load cluster descriptors.
- [ ] Load all geometry pages.
- [ ] Load fallback mesh.
- [ ] Register asset in RVG registry.
- [ ] Reject corrupted assets safely.

10.2 GPU metadata buffers

Create:

- RVG asset table.
- Hierarchy-node buffer.
- Cluster descriptor buffer.
- Page descriptor buffer.
- Material-slot buffer.
- Resident geometry buffer.

Tasks:

- [ ] Define CPU/GPU layouts.
- [ ] Add alignment assertions.

- [ ] Upload metadata.
- [ ] Add generation IDs.
- [ ] Add asset offsets.
- [ ] Add capacity handling.

10.3 RVG GPU instance data

Extend instance record with:

- RVG asset index.
- Root hierarchy node.
- Material override offset.
- Instance scale factor.
- Flags.
- Mirrored-transform state.

10.4 Initial root-only rendering

- [ ] Select root representation.
- [ ] Append root clusters.
- [ ] Build indirect cluster draws.
- [ ] Render depth.
- [ ] Render unlit color.
- [ ] Render cluster ID.
- [ ] Render asset ID.
- [ ] Render object ID.
- [ ] Compare with fallback silhouette.

10.5 Cluster-level frustum culling

- [ ] Transform cluster sphere.
- [ ] Frustum-test cluster.
- [ ] Append visible cluster.
- [ ] Record culling reason.
- [ ] Guard work-list capacity.

10.6 Normal-cone culling

- [ ] Decode normal cone.
- [ ] Transform cone correctly.
- [ ] Account for mirrored transforms.
- [ ] Disable cone culling for unsafe clusters.
- [ ] Add conservative cutoff bias.
- [ ] Add cone debug visualization.
- [ ] Compare with cone culling disabled.

10.7 Cluster indirect rendering

- [ ] Build one indirect command per selected cluster.
- [ ] Resolve cluster geometry offsets.

- [ ] Resolve instance ID.
- [ ] Resolve material slot.
- [ ] Render with shared buffers.
- [ ] Add cluster draw statistics.
- [ ] Add capacity protection.

G7 exit gates

- [ ] Root geometry renders for multiple assets.
- [ ] Multiple RVG instances render.
- [ ] Cluster frustum culling works.
- [ ] Cone culling produces no missing front-facing geometry.
- [ ] Object and cluster ID modes work.
- [ ] Conventional fallback can replace RVG at runtime.
- [ ] All geometry remains resident; no streaming is active yet.

---

11. G8 — Hierarchical LOD Traversal

Objective

Select cluster detail automatically using projected geometric error.

Dependencies

G7.

11.1 Candidate-node buffers

Create:

- Current candidate nodes.
- Next candidate nodes.
- Selected nodes.
- Selected clusters.
- Traversal counters.
- Overflow state.

11.2 Projected error calculation

Implement:

```
projectionScale =
viewportHeight / (2 × tan(verticalFOV / 2))

projectedError =
objectSpaceError
× maximumInstanceScale
```

```
× projectionScale
/ max(viewDepth, epsilon)
```

Tasks:

- [ ] Handle perspective projection.
- [ ] Handle orthographic projection.
- [ ] Handle near-plane intersection.
- [ ] Handle extremely small depth.
- [ ] Handle non-uniform scale conservatively.
- [ ] Add LOD bias.
- [ ] Add target pixel error.
- [ ] Add maximum traversal depth.

11.3 Hierarchy traversal

For each node:

- [ ] Frustum-test node bounds.
- [ ] Calculate projected error.
- [ ] Select node if error is acceptable.
- [ ] Otherwise enqueue children.
- [ ] Select leaf when no children exist.
- [ ] Guard candidate capacity.
- [ ] Record hierarchy depth.
- [ ] Record selection reason.
- [ ] Prevent infinite traversal caused by corrupted metadata.

11.4 Crack prevention

- [ ] Verify parent and children cover identical region.
- [ ] Enforce group-based transitions.
- [ ] Prevent arbitrary neighbouring cluster LOD changes.
- [ ] Validate boundary vertices.
- [ ] Add transition stress camera.
- [ ] Add wireframe transition view.
- [ ] Add depth difference test.

11.5 LOD debugging

Add modes:

- Hierarchy level.
- Geometric error.
- Projected error.
- Selected nodes.
- Parent/child boundaries.
- Force root.
- Force full detail.
- Freeze selection.

- LOD bias.
- Pixel-error threshold.

G8 exit gates

- [ ] LOD changes automatically with distance.
- [ ] No author-created LOD meshes are needed.
- [ ] No visible cracks occur during transitions.
- [ ] Force-root and force-full-detail work.
- [ ] Triangle count decreases with distance.
- [ ] Traversal remains within configured capacity.

---

12. G9 — Virtual Geometry Page Streaming

Objective

Stream detailed geometry into a fixed-size GPU page cache while preserving root fallback.

Dependencies

G8.

12.1 Residency state machine

Implement:

```
enum class GeometryPageState : uint8_t
{
    Unloaded,
    Requested,
    Reading,
    Decompressing,
    UploadQueued,
    Resident,
    EvictionPending,
    Failed
};
```

- [ ] Validate legal state transitions.
- [ ] Add state timestamps.
- [ ] Add retry count.
- [ ] Add failure reason.
- [ ] Add thread-safe transitions.
- [ ] Add debug state names.

12.2 Physical GPU page pool

- [ ] Allocate fixed-size GPU page cache.
- [ ] Divide into physical page slots.
- [ ] Reserve root-page region.
- [ ] Implement physical page allocator.
- [ ] Add generation counter.
- [ ] Add free list.
- [ ] Add deferred page reuse.
- [ ] Add cache usage statistics.

12.3 Virtual page table

```
struct GPUPageMapping
{
    uint32_t physicalPage;
    uint32_t generation;
    uint32_t flags;
    uint32_t reserved;
};
```

Tasks:

- [ ] Create one mapping per virtual page.
- [ ] Represent non-resident page.
- [ ] Mark root page permanently resident.
- [ ] Update mapping only after upload completion.
- [ ] Add generation validation.
- [ ] Add double-buffered or staged mapping updates.
- [ ] Prevent traversal from reading partially updated mappings.

12.4 Residency-aware traversal

When a desired child page is absent:

- [ ] Generate page request.
- [ ] Render current resident ancestor.
- [ ] Continue traversing other independent nodes.
- [ ] Never output an empty region.
- [ ] Record fallback depth.
- [ ] Record missing-detail statistics.

12.5 GPU request generation

Create:

```
struct GeometryStreamingRequest
{
```

```
    uint32_t assetID;
    uint32_t pageID;
    float projectedImportance;
    uint16_t viewMask;
    uint16_t flags;
};
```

Tasks:

- [ ] Append missing pages.
- [ ] Deduplicate repeated requests.
- [ ] Accumulate requesting instance count.
- [ ] Record maximum projected importance.
- [ ] Guard request capacity.
- [ ] Add request readback.
- [ ] Use multi-frame buffering to avoid stalls.

12.6 Streaming scheduler

Priority factors:

- Projected error.
- Main-view importance.
- Number of requesting instances.
- Current ancestor quality.
- Request age.
- Camera distance.
- Selected-object priority.
- Shadow-view priority later.

Tasks:

- [ ] Sort or bucket requests.
- [ ] Apply I/O budget.
- [ ] Apply decompression budget.
- [ ] Apply upload budget.
- [ ] Prevent one asset monopolizing bandwidth.
- [ ] Add request ageing.
- [ ] Cancel obsolete requests where safe.

12.7 Async file I/O

- [ ] Open page ranges asynchronously.
- [ ] Batch nearby reads.
- [ ] Add aligned read buffers.
- [ ] Add I/O worker queue.
- [ ] Handle file close during scene unload.
- [ ] Handle missing file.
- [ ] Handle checksum failure.
- [ ] Add read latency statistics.

- [ ] Add slow-disk simulation.

12.8 Decompression

- [ ] Create decompression worker pool.
- [ ] Validate output size.
- [ ] Validate checksum.
- [ ] Decode page payload.
- [ ] Handle decompression failure.
- [ ] Return failed page to safe state.
- [ ] Preserve ancestor fallback.
- [ ] Add decompression timing.

12.9 GPU upload

- [ ] Reserve physical page.
- [ ] Reserve staging space.
- [ ] Copy decompressed payload.
- [ ] Submit transfer.
- [ ] Signal timeline semaphore.
- [ ] Update mapping after completion.
- [ ] Mark page resident.
- [ ] Prevent physical-page reuse while upload is in flight.

12.10 Eviction

- [ ] Track last-used frame.
- [ ] Track current request count.
- [ ] Track page dependencies.
- [ ] Protect root pages.
- [ ] Protect recently uploaded pages.
- [ ] Protect in-flight pages.
- [ ] Apply hysteresis.
- [ ] Evict low-priority old pages.
- [ ] Invalidate page table.
- [ ] Defer physical-page reuse until safe.

12.11 Streaming stress tests

- [ ] Rapid camera teleport.
- [ ] Camera oscillation.
- [ ] Very small cache.
- [ ] Slow storage.
- [ ] Random read failure.
- [ ] Corrupted page.
- [ ] Asset unload while streaming.
- [ ] Scene reload.
- [ ] Hundreds of instances sharing pages.
- [ ] Multiple different RVG assets.

G9 exit gates

- [ ] Detailed pages stream during camera movement.
- [ ] No holes appear while pages are missing.
- [ ] Root geometry is always available.
- [ ] Cache budget is respected.
- [ ] Eviction does not cause crashes or stale mappings.
- [ ] Failed pages produce reduced quality, not missing geometry.
- [ ] Streaming statistics are visible.

---

13. G10 — Visibility Buffer Rendering

Objective

Rasterize geometry identity and depth instead of full material GBuffer data.

Dependencies

G8. Streaming from G9 should be integrated before production completion.

13.1 Visibility attachments

Create:

- `VisibilityInstance`
- `VisibilityCluster`
- `VisibilityPrimitive`
- `SceneDepth`

Initial formats:

```
VisibilityInstance  = R32_UINT
VisibilityCluster   = R32_UINT
VisibilityPrimitive = R32_UINT
SceneDepth          = D32_FLOAT
```

Tasks:

- [ ] Define invalid pixel value.
- [ ] Clear all attachments.
- [ ] Add resize support.
- [ ] Add frame-graph declarations.
- [ ] Add attachment inspector.
- [ ] Avoid premature bit packing.

13.2 Visibility vertex/raster shader

- [ ] Resolve instance transform.
- [ ] Resolve cluster payload.
- [ ] Decode positions.
- [ ] Emit clip-space position.
- [ ] Output flat instance ID.
- [ ] Output flat cluster ID.
- [ ] Output primitive ID.
- [ ] Write exact scene depth.
- [ ] Handle front-face orientation.
- [ ] Handle mirrored instances.
- [ ] Handle near-plane clipping.

13.3 Picking integration

- [ ] Decode instance ID under cursor.
- [ ] Resolve entity ID.
- [ ] Validate generation.
- [ ] Support RVG and conventional objects.
- [ ] Add cluster/primitive inspection.
- [ ] Add editor selected-cluster view.

13.4 Visibility comparison

- [ ] Compare RVG depth against conventional fallback depth.
- [ ] Produce depth difference view.
- [ ] Compare silhouettes.
- [ ] Compare object IDs.
- [ ] Test thin triangles.
- [ ] Test near-plane triangles.
- [ ] Test mirrored transforms.

G10 exit gates

- [ ] Every visible RVG pixel has valid identity.
- [ ] Object picking works.
- [ ] Depth matches the selected hierarchy geometry.
- [ ] No background pixel contains stale identity.
- [ ] Visibility attachments are inspectable.
- [ ] Conventional fallback can be overlaid for comparison.

---

14. G11 — Attribute Reconstruction

Objective

Reconstruct surface attributes from visibility-buffer identity.

Dependencies

G10.

14.1 Visibility decoding

For each pixel:

- [ ] Read instance ID.
- [ ] Read cluster ID.
- [ ] Read primitive ID.
- [ ] Validate IDs.
- [ ] Resolve RVG asset.
- [ ] Resolve page mapping.
- [ ] Resolve cluster descriptor.
- [ ] Resolve micro-triangle indices.
- [ ] Resolve three vertices.

14.2 Position reconstruction

- [ ] Decode quantized positions.
- [ ] Transform to world space.
- [ ] Transform to view space.
- [ ] Recalculate clip-space positions.
- [ ] Calculate screen-space triangle.
- [ ] Calculate barycentric coordinates.
- [ ] Apply perspective correction.
- [ ] Handle degenerate projected triangles.

14.3 Attribute interpolation

Reconstruct:

- [ ] World position.
- [ ] Geometric normal.
- [ ] Vertex normal.
- [ ] Tangent.
- [ ] Bitangent sign.
- [ ] UV0.
- [ ] UV1 if supported.
- [ ] Vertex color.
- [ ] Material slot.
- [ ] Current clip position.
- [ ] Previous clip position.
- [ ] Motion vector.

14.4 Normal transform correctness

- [ ] Support non-uniform scale.
- [ ] Support negative scale.

- [ ] Normalize interpolated normal.
- [ ] Reconstruct tangent frame.
- [ ] Preserve mirrored UV orientation.
- [ ] Add reconstructed-normal debug mode.

14.5 Reconstruction validation

Compare against conventional interpolation:

- [ ] World position difference.
- [ ] Normal angle difference.
- [ ] UV difference.
- [ ] Motion-vector difference.
- [ ] Material-slot difference.
- [ ] Depth difference.

Test:

- Flat surfaces.
- Curved surfaces.
- Hard edges.
- UV seams.
- Mirrored UVs.
- Non-uniform scale.
- Negative scale.
- Long thin triangles.
- Subpixel triangles.

G11 exit gates

- [ ] Reconstructed UVs match the conventional path.
- [ ] Reconstructed normals remain within tolerance.
- [ ] Material slots resolve correctly.
- [ ] Motion vectors are stable.
- [ ] No NaN/Inf values enter the GBuffer.
- [ ] Reconstruction debug modes are available.

---

15. G12 — Material Binning and Deferred Resolve

Objective

Evaluate materials only for visible pixels and write the standard Radiance GBuffer.

Dependencies

G11 and the existing Radiance material system.

15.1 Material lookup

- [ ] Resolve source material slot.
- [ ] Apply per-instance override.
- [ ] Resolve final material ID.
- [ ] Validate material generation.
- [ ] Route missing material to fallback.
- [ ] Route unsupported material to conventional renderer.

15.2 Material bin count pass

- [ ] Read visibility pixels.
- [ ] Ignore invalid pixels.
- [ ] Resolve material ID.
- [ ] Atomically increment material count.
- [ ] Guard material-table bounds.
- [ ] Record unsupported material count.

15.3 Prefix-sum pass

- [ ] Calculate bin offsets.
- [ ] Calculate total material pixels.
- [ ] Validate capacity.
- [ ] Clear per-material scatter counters.
- [ ] Add GPU prefix-sum implementation.
- [ ] Add CPU fallback for debugging.

15.4 Pixel scatter pass

```
struct MaterialPixel
{
    uint16_t x;
    uint16_t y;
};
```

- [ ] Resolve material bin.
- [ ] Append pixel coordinate.
- [ ] Guard list capacity.
- [ ] Record overflow.
- [ ] Support large viewport dimensions safely.

15.5 Material evaluation

For each material bin:

- [ ] Dispatch material evaluator.
- [ ] Reconstruct attributes.
- [ ] Load material constants.
- [ ] Sample base-color texture.

- [ ] Sample normal map.
- [ ] Sample metallic/roughness map.
- [ ] Sample AO.
- [ ] Sample emissive.
- [ ] Apply UV scale/offset.
- [ ] Write GBuffer albedo.
- [ ] Write GBuffer normal.
- [ ] Write GBuffer material properties.
- [ ] Write material ID.
- [ ] Write velocity if enabled.

15.6 Initially supported material features

- [ ] Opaque PBR.
- [ ] Multiple material slots.
- [ ] Base color.
- [ ] Normal map.
- [ ] Metallic.
- [ ] Roughness.
- [ ] Ambient occlusion.
- [ ] Emissive.
- [ ] UV transform.
- [ ] Per-instance material override.

15.7 Deferred features

Keep on conventional path initially:

- Transparency.
- Refraction.
- Complex masked materials.
- Pixel depth offset.
- Runtime displacement.
- Custom vertex deformation.
- Multi-layer materials.
- Subsurface shading.
- Hair shading.

15.8 GBuffer parity testing

- [ ] Compare albedo.
- [ ] Compare normals.
- [ ] Compare metallic.
- [ ] Compare roughness.
- [ ] Compare AO.
- [ ] Compare emissive.
- [ ] Compare material ID.
- [ ] Compare final deferred lighting.

G12 exit gates

- [ ] RVG writes the normal Radiance GBuffer.
- [ ] Deferred lighting requires no RVG-specific lighting path.
- [ ] Supported RVG materials match conventional materials.
- [ ] Unsupported materials route safely.
- [ ] Material bin overflow is detectable.
- [ ] Material resolve timings are visible.

---

16. G13 — Mesh Shader Backend

Objective

Add mesh-shader cluster rasterization on supported hardware.

Dependencies

G10–G12 stable hardware indexed path.

Tasks

- [ ] Detect mesh shader capability.
- [ ] Create mesh shader pipeline.
- [ ] Define one cluster per mesh workgroup where practical.
- [ ] Decode cluster payload in mesh shader.
- [ ] Emit vertices.
- [ ] Emit primitive indices.
- [ ] Output visibility IDs.
- [ ] Preserve exact depth convention.
- [ ] Handle cluster vertex/triangle limits.
- [ ] Add task-shader amplification optionally.
- [ ] Add mesh-shader statistics.
- [ ] Compare indexed and mesh-shader outputs.
- [ ] Keep indexed path as fallback.
- [ ] Add runtime backend switch.

G13 exit gates

- [ ] Mesh shader and indexed paths produce matching visibility buffers.
- [ ] Unsupported GPUs automatically use indexed rendering.
- [ ] Engine startup does not require mesh-shader support.
- [ ] Performance difference is measurable in the profiler.

---

17. G14 — Software Micro-Triangle Rasterizer

Objective

Rasterize clusters dominated by subpixel triangles using compute shaders.

Dependencies

Complete and stable hardware visibility path.

17.1 Raster classification

Calculate:

- Projected cluster area.
- Estimated pixel count.
- Triangle count.
- Pixels per triangle.
- Near-plane intersection.
- Hardware raster suitability.

Tasks:

- [ ] Define configurable crossover threshold.
- [ ] Classify hardware clusters.
- [ ] Classify software clusters.
- [ ] Add hysteresis to classification.
- [ ] Add classification debug colors.

17.2 Tile binning

- [ ] Divide screen into tiles.
- [ ] Project cluster bounds.
- [ ] Determine overlapping tiles.
- [ ] Append cluster to tile list.
- [ ] Guard tile-list capacity.
- [ ] Record binning overflow.
- [ ] Add tile occupancy visualization.

17.3 Triangle setup

- [ ] Decode vertices.
- [ ] Transform to clip space.
- [ ] Clip against near plane.
- [ ] Convert to screen space.
- [ ] Calculate edge equations.
- [ ] Determine winding.
- [ ] Calculate depth plane.
- [ ] Reject degenerate triangles.

17.4 Coverage and depth

- [ ] Test pixel/sample coverage.
- [ ] Apply top-left fill rule.
- [ ] Calculate perspective-correct depth.
- [ ] Perform atomic depth operation.
- [ ] Write visibility identity on depth win.
- [ ] Handle ties deterministically.
- [ ] Preserve compatibility with hardware raster depth.

17.5 Hardware/software merge

- [ ] Share or reconcile depth representation.
- [ ] Ensure one winning primitive.
- [ ] Merge visibility outputs.
- [ ] Prevent race conditions.
- [ ] Add output difference view.
- [ ] Compare hardware-only and hybrid results.

G14 exit gates

- [ ] Hybrid output matches hardware-only output.
- [ ] No cracks appear between hardware and software clusters.
- [ ] Subpixel-heavy scenes show measurable improvement.
- [ ] Software raster can be disabled globally.
- [ ] All work-list overflows are reported.

---

18. G15 — Shadows and Multi-View Traversal

Objective

Reuse RVG hierarchy traversal for shadow and auxiliary views.

Dependencies

G9–G12.

Tasks

- [ ] Generalize view data.
- [ ] Add view ID and view mask.
- [ ] Support main camera.
- [ ] Support directional shadow cascades.
- [ ] Support spot-light shadow view.
- [ ] Apply view-specific pixel-error thresholds.
- [ ] Apply shadow-view page priority.
- [ ] Avoid duplicating identical page requests.

- [ ] Share resident pages across views.
- [ ] Add per-view traversal statistics.
- [ ] Add shadow-cluster visualizer.
- [ ] Validate shadow silhouette against conventional path.
- [ ] Keep transparent and skeletal shadow casters conventional.

G15 exit gates

- [ ] RVG geometry casts directional shadows.
- [ ] Shadow traversal respects its own detail threshold.
- [ ] Main-view quality is not starved by shadow requests.
- [ ] Multi-view page requests are deduplicated.
- [ ] Shadow geometry has no missing regions.

---

19. G16 — Production Hardening

Objective

Make RVG safe for long development sessions and production content.

19.1 Failure recovery

- [ ] Corrupted asset header.
- [ ] Corrupted hierarchy node.
- [ ] Corrupted page checksum.
- [ ] Missing page file.
- [ ] Failed decompression.
- [ ] Failed GPU upload.
- [ ] Out-of-memory.
- [ ] Buffer overflow.
- [ ] Invalid material.
- [ ] Stale instance.
- [ ] Device loss.
- [ ] Renderer recreation.
- [ ] Scene unload during I/O.

Every failure must result in:

- Reduced detail.
- Root fallback.
- Conventional fallback.
- Clear diagnostic.

It must not result in:

- Crash.
- GPU hang.
- Out-of-bounds write.
- Permanent hole.

- Stale geometry from another asset.

19.2 Hot reload

- [ ] Recook asset.
- [ ] Detect asset version change.
- [ ] Load new metadata.
- [ ] Keep old asset active until replacement is ready.
- [ ] Swap registry generation.
- [ ] Invalidate old page requests.
- [ ] Defer old resource destruction.
- [ ] Update all instances.
- [ ] Preserve material overrides where compatible.

19.3 Device recreation

- [ ] Recreate geometry page pool.
- [ ] Recreate metadata buffers.
- [ ] Recreate page table.
- [ ] Recreate visibility targets.
- [ ] Recreate pipelines.
- [ ] Restore root pages first.
- [ ] Restore detailed residency progressively.
- [ ] Preserve CPU asset registry.

19.4 Performance telemetry

Record:

- GPU scene upload time.
- Instance culling time.
- HZB build time.
- Hierarchy traversal time.
- Cluster culling time.
- Request generation time.
- Raster time.
- Material bin time.
- Material resolve time.
- Page reads per frame.
- Bytes read.
- Bytes decompressed.
- Bytes uploaded.
- Cache hit rate.
- Eviction count.
- Root fallback count.
- Selected triangle count.
- Visible triangle count.
- Pixels per selected triangle.

19.5 Automated tests

Unit tests

- Geometry handle generations.
- Arena allocation.
- Arena freeing.
- Page state transitions.
- File parser.
- Checksum validation.
- Hierarchy monotonicity.
- Cluster decode.
- Quantization error.
- Material resolution.
- Request deduplication.

GPU tests

- Frustum culling.
- HZB culling.
- Hierarchy traversal.
- Work-list capacity.
- Indirect command generation.
- Visibility output.
- Attribute reconstruction.
- Prefix sum.
- Software depth.

Integration tests

- Scene load/unload.
- Asset hot reload.
- Camera teleport.
- Cache exhaustion.
- Slow storage.
- Corrupted page.
- Device recreation.
- Multiple assets.
- Thousands of instances.
- Editor play-mode transition.

Golden-image tests

- Root-only.
- Full detail.
- Multiple LOD distances.
- Streaming transition.
- Material resolve.
- Negative scale.
- UV seams.
- Shadow output.
- Hardware/software raster parity.

G16 exit gates

- [ ] RVG survives extended editor sessions.
- [ ] Asset reload is safe.
- [ ] Scene unload during streaming is safe.
- [ ] Cache exhaustion preserves visible geometry.
- [ ] Corrupted pages fall back safely.
- [ ] Device recreation restores rendering.
- [ ] Automated test suite passes.
- [ ] Performance telemetry is available in the editor.

---

20. Required RVG Debug Panel

Create an editor panel named:

```
Radiance Virtual Geometry
```

Overview

Display:

- RVG enabled.
- Active capability tier.
- Active raster backend.
- Loaded RVG assets.
- RVG instances.
- Visible instances.
- Traversed hierarchy nodes.
- Selected clusters.
- Selected triangles.
- Visible pixels.

Culling

Display:

- Instance frustum culled.
- Instance occlusion culled.
- Node frustum culled.
- Cluster frustum culled.
- Cluster cone culled.
- Cluster occlusion culled.

Residency

Display:

- Virtual pages.
- Resident pages.
- Root pages.
- Requested pages.
- Reading pages.
- Decompressing pages.
- Uploading pages.
- Failed pages.
- Cache usage.
- Cache hit rate.
- Bytes read.
- Bytes uploaded.
- Evictions.

Overflow

Display:

- Candidate-node overflow.
- Selected-cluster overflow.
- Streaming-request overflow.
- Indirect-command overflow.
- Material-bin overflow.
- Tile-bin overflow.
- Software-raster overflow.

Controls

- Enable RVG.
- Force conventional rendering.
- Force root geometry.
- Force full detail.
- Freeze LOD.
- Freeze culling.
- Freeze HZB.
- Freeze streaming.
- Disable eviction.
- Set target pixel error.
- Set LOD bias.
- Set cache budget.
- Set upload budget.
- Set request budget.
- Select raster backend.

---

21. Required Visualization Modes

- [ ] Conventional versus RVG.
- [ ] Asset ID.
- [ ] Instance ID.
- [ ] Cluster ID.
- [ ] Primitive ID.
- [ ] Hierarchy level.
- [ ] Cluster group.
- [ ] Geometric error.
- [ ] Projected error.
- [ ] Selected LOD.
- [ ] Root fallback.
- [ ] Resident pages.
- [ ] Missing pages.
- [ ] Streaming requests.
- [ ] Page age.
- [ ] Cache location.
- [ ] Frustum-culled geometry.
- [ ] Cone-culled geometry.
- [ ] Occlusion-culled geometry.
- [ ] Hardware raster.
- [ ] Mesh shader raster.
- [ ] Software raster.
- [ ] Visibility instance.
- [ ] Visibility cluster.
- [ ] Visibility primitive.
- [ ] Reconstructed position.
- [ ] Reconstructed normal.
- [ ] Reconstructed tangent.
- [ ] Reconstructed UV.
- [ ] Material bins.
- [ ] Overflow status.

---

22. Recommended Source Structure

```
Rendering/
└── Geometry/
    ├── Core/
    │   ├── GeometryHandle.*
    │   ├── GeometryTypes.*
    │   ├── GeometryCapabilities.*
    │   └── GeometryRouting.*
    │
    ├── Assets/
    │   ├── MeshAsset.*
    │   ├── MeshRuntime.*
```

```
│   ├── VirtualGeometryAsset.*
│   ├── VirtualGeometryRegistry.*
│   └── RVGFileFormat.*
│
├── Arena/
│   ├── GeometryArena.*
│   ├── GeometryAllocator.*
│   ├── GeometryUploader.*
│   ├── GeometryGarbageCollector.*
│   └── GeometryArenaStats.*
│
├── GPUScene/
│   ├── GPUScene.*
│   ├── GPUSceneInstanceAllocator.*
│   ├── GPUSceneUploader.*
│   ├── GPUMeshTable.*
│   └── GPUMaterialOverrideTable.*
│
├── Virtual/
│   ├── RVGHierarchyNode.*
│   ├── RVGCluster.*
│   ├── RVGPage.*
│   ├── RVGPageTable.*
│   ├── RVGPhysicalPagePool.*
│   ├── RVGResidencyManager.*
│   ├── RVGStreamingScheduler.*
│   └── RVGRequestReadback.*
│
├── Passes/
│   ├── GeometryInstanceCullPass.*
│   ├── HZBBuildPass.*
│   ├── RVGHierarchyTraversePass.*
│   ├── RVGClusterCullPass.*
│   ├── RVGPageRequestPass.*
│   ├── RVGVisibilityPass.*
│   ├── RVGMaterialBinPass.*
│   └── RVGMaterialResolvePass.*
│
├── Raster/
│   ├── IndexedClusterRasterizer.*
│   ├── MeshShaderClusterRasterizer.*
│   ├── SoftwareClusterRasterizer.*
│   └── RasterPathClassifier.*
│
├── Fallback/
│   ├── ConventionalGeometryRenderer.*
│   ├── FallbackMeshRenderer.*
│   └── UnsupportedMaterialRouter.*
│
└── Debug/
```

```
        ├── RVGDebugPanel.*
        ├── RVGVisualizer.*
        ├── RVGRenderStats.*
        ├── RVGOverflowReporter.*
        └── RVGCaptureTools.*

Tools/
└── RVGCooker/
    ├── RVGCookerMain.*
    ├── MeshCanonicalizer.*
    ├── MeshValidator.*
    ├── MeshAdjacency.*
    ├── MaterialPartitioner.*
    ├── ClusterBuilder.*
    ├── ClusterGroupBuilder.*
    ├── ClusterSimplifier.*
    ├── HierarchyBuilder.*
    ├── GeometricErrorCalculator.*
    ├── RootGeometryBuilder.*
    ├── FallbackMeshBuilder.*
    ├── RVGPagePacker.*
    ├── RVGCompressor.*
    ├── RVGWriter.*
    └── RVGCookerReport.*
```

---

23. Recommended Implementation Sequence

Implement in this exact order:

```
1. Geometry contracts
2. Stable CPU rendering
3. Shared GeometryArena
4. Persistent GPUScene
5. GPU frustum culling
6. Correct indexed indirect rendering
7. HZB occlusion
8. RVG cooker
9. Root cluster rendering
10. All-resident hierarchy traversal
11. Projected-error LOD
12. Geometry page cache
13. Streaming requests
14. Async I/O and decompression
15. Residency-aware fallback
16. Visibility buffer
17. Attribute reconstruction
18. Material binning
```

```
19. Deferred material resolve
20. Mesh shader backend
21. Software micro-rasterization
22. Shadow views
23. Production hardening
```

Do not implement these prematurely:

- Software rasterization before visibility-buffer correctness.
- Geometry streaming before all-resident hierarchy rendering.
- Hierarchical LOD before the cooker produces valid parent geometry.
- Mesh shaders before indexed cluster rendering works.
- Virtual materials before the standard material system works.
- Shadow traversal before the main-view traversal is stable.
- Runtime deformation inside RVG v1.

---

24. Final Completion Criteria

Radiance Virtual Geometry v1 is complete only when:

- [ ] Conventional rendering remains available.
- [ ] Static opaque assets can be imported as RVG.
- [ ] High-poly assets cook deterministically.
- [ ] Assets contain cluster hierarchies.
- [ ] Detail is selected by projected geometric error.
- [ ] Root geometry is permanently resident.
- [ ] Detailed pages stream asynchronously.
- [ ] Missing pages never create holes.
- [ ] GPU cache budget is enforced.
- [ ] Frustum culling works.
- [ ] Cone culling works.
- [ ] HZB occlusion works.
- [ ] Visibility buffer contains valid identities.
- [ ] Attributes reconstruct correctly.
- [ ] RVG materials write the normal Radiance GBuffer.
- [ ] Deferred lighting treats RVG like conventional geometry.
- [ ] Object picking works.
- [ ] RVG casts shadows.
- [ ] Unsupported materials use the fallback renderer.
- [ ] Unsupported hardware uses the conventional renderer.
- [ ] Buffer overflows are detected.
- [ ] Corrupted pages recover safely.
- [ ] Device recreation is supported.
- [ ] Debug views expose every major stage.
- [ ] Automated parity and stress tests pass.
- [ ] RVG improves large-scene performance without sacrificing correctness.